

Raumtypen und Buchungsterminal

Organisation: Sie arbeiten im selben Zweierteam und im selben Repository wie in Blatt 1, mit den Rollen A und B. Steht vor einem Arbeitsschritt (A) oder (B), erledigt ihn nur diese Person. Das Testat schreibt jede Person allein.

Abgabe: ein ZIP des Projektordners einschließlich des Ordners `.git` über Moodle/APA bis [OFFEN: Datum]. Der letzte Commit auf main trägt den Tag `abgabe-2`.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Alle Räume, Gruppen, Zeiten und Regeln sind weiterhin frei erfunden und beschreiben keine realen Räume oder Buchungsregeln der THI.

Erste Aufgabe (15 Minuten): Laden Sie `freiraum-start-2.zip` aus Moodle. Der Startstand ist der Referenzstand nach Blatt 1; dieses Blatt, Testat 2 und die Prüfprogramme verwenden seine Klassen- und Methodennamen.

- Spielen Sie den Startstand im Branch `start-2` ein, wie es das `README.md` des Startstands beschreibt. Lesen Sie vor dem Commit mit `git diff --staged`, worin sich Ihr Stand und der Startstand unterscheiden.
- Entscheiden Sie gemeinsam: Entweder mergen Sie `start-2` nach `main`, oder Sie führen Ihren eigenen Stand weiter und benennen abweichende Methoden nach dem Startstand um (`getGruppe`, `getPersonen`, `getZeitfenster`, `dauerMinuten`, `ueberschneidet`, `istFrei`, `buchen`, `getKennung`). Schreiben Sie die Entscheidung mit ihrem Grund in die Commit-Nachricht.
- `PruefungFormat`, `PruefungZeitfenster`, `PruefungBuchung` und `PruefungRaum` enden auf `main` mit `0 FEHLER`.

Zweite Aufgabe (45 Minuten): Jede Person arbeitet in ihrem Branch, (A) in `a-nummer`, (B) in `b-fabrik`.

- (A) Jede gültige Buchung erhält eine fortlaufende Nummer. Buchung bekommt das Attribut `nummer`, den Zähler `private static int naechsteNummer = 1` und den Getter `getNummer()`. Der Konstruktor vergibt die Nummer erst nach allen Prüfungen, damit eine abgewiesene Buchung keine Nummer verbraucht. `toString()` beginnt mit der Nummer: `#1 Di 10:00-11:30 Lerngruppe Statistik, 4 Pers.`
- (A) Erzeugen Sie in einem kleinen `main` drei Buchungen und geben Sie ihre Nummern aus. Entfernen Sie dann `testweise static` vor `naechsteNummer` und starten Sie erneut. Tragen Sie beide Ausgaben unter „Blatt 2“ in `NOTIZEN.md` ein, erklären Sie den Unterschied in einem Satz und setzen Sie `static` wieder ein.
- (B) Zeitfenster erhält die Fabrikmethode `public static Zeitfenster aus(String tag, String beginn, String ende)`. Sie wandelt den Tag mit `Integer.parseInt` und beide Uhrzeiten mit `Format.minuten` um und ruft den Konstruktor auf. `Zeitfenster.aus("2", "10:00", "11:30")` liefert also dasselbe Fenster wie `new Zeitfenster(2, 600, 690)`. Die `NumberFormatException` bei `"10:00"` fängt aus nicht ab.
- (B) Beantworten Sie in `NOTIZEN.md`: Warum muss `aus` `static` sein? Warum bleiben die Prüfungen der Invarianten im Konstruktor, statt in `aus` zu wandern?

Öffnen Sie je einen Pull Request, lesen Sie den Code der anderen Person und mergen Sie beide nach `main`, bevor die dritte Aufgabe beginnt.

Dritte Aufgabe (60 Minuten): Schreiben Sie zuerst gemeinsam auf `main`: Raum erhält die Methode `public int maxDauerMinuten()`. Ein gewöhnlicher Raum darf den ganzen Tag gebucht werden, die Methode liefert also

Zeitfenster.SPAETESTENS - Zeitfenster.FRUEHESTENS, das sind 900 Minuten. buchen lehnt zusätzlich jede Buchung ab, die länger dauert als `maxDauerMinuten()`. Committen und pushen Sie, dann legt jede Person ihren Branch an, (A) a-lernraum, (B) b-labor.

- (A) Lernraum extends Raum mit dem Konstruktor `Lernraum(String kennung, int plaetze).maxDauerMinuten()` liefert 180, `toString()` liefert "Lernraum " + `super.toString()`.
- (B) Buchung erhält das Attribut einweisung mit dem Getter `hatEinweisung()` und einen zweiten Konstruktor `Buchung(String gruppe, int personen, Zeitfenster zeitfenster, boolean einweisung)`. Der bisherige Konstruktor ruft ihn mit `this(gruppe, personen, zeitfenster, false)` auf, damit die Prüfungen nur einmal dastehen.
- (B) Labor extends Raum mit dem Konstruktor `Labor(String kennung, int plaetze).maxDauerMinuten()` liefert 240. `buchen` liefert für eine Buchung ohne Einweisung sofort `false` und überlässt alle anderen Prüfungen der Fassung aus Raum. `toString()` liefert "Labor " + `super.toString()`.

Setzen Sie vor jede Methode, die eine Methode aus Raum ersetzt, die Annotation `@Override`. Fehlt im Konstruktor der Aufruf `super(kennung, plaetze)`, meldet der Compiler:

```
src/main/java/freiraum/Lernraum.java:6: error: constructor Raum in class Raum cannot be applied to given types;
    public Lernraum(String kennung, int plaetze) {
                                   ^
    required: String,int
    found:    no arguments
    reason: actual and formal argument lists differ in length
1 error
```

Auf einem deutsch eingestellten Rechner erscheint die Meldung übersetzt. Erklären Sie sich gegenseitig, woher die „no arguments“ in der Meldung kommen. Pull Requests, mergen.

Vierte Aufgabe (40 Minuten): Zwei Zeitfenster mit denselben Werten sollen als gleich gelten, auch wenn sie mit zwei verschiedenen `new` erzeugt wurden.

- (B) schreibt zuerst eine neue Klasse `PruefungRaumtypen` mit Fällen zur Gleichheit, ohne den Code von A anzusehen: Zwei getrennt erzeugte Fenster mit denselben Werten sind `equals`, aber nicht `==`; sie haben denselben `hashCode`; ein `HashSet<Zeitfenster>`, das das eine enthält, findet mit `contains` das andere; ein anderer Tag oder ein anderes Ende ergibt ungleich; `equals(null)` und der Vergleich mit einem `String` liefern `false`.
- (A) überschreibt in `Zeitfenster` die Methoden `equals(Object o)` und `hashCode()`.
- (A) schreibt testweise `@Override public boolean equals(Zeitfenster z)`. Der Compiler meldet `error: method does not override or implement a method from a supertype`. Tragen Sie in `NOTIZEN.md` ein, was `@Override` hier verhindert hat, und stellen Sie `equals(Object o)` wieder her.
- (B) kommentiert `hashCode()` testweise aus, startet `PruefungRaumtypen` und notiert, welche Fälle jetzt scheitern und warum. Danach kommt `hashCode()` zurück.

Fünfte Aufgabe (80 Minuten): Bauen Sie ein Buchungsterminal für die Konsole.

- (A) Campus verwaltet höchstens zehn Räume in einem Array `Raum[] raeume` mit `anzahl`. `boolean hinzufuegen(Raum raum)` liefert `false`, wenn der Campus voll oder die Kennung schon vergeben ist. `Raum finde(String kennung)`

liefert null für eine unbekannte Kennung. `void raumeAusgeben()` druckt je Raum eine Zeile mit `toString()` und Höchstdauer.

- (B) Buchungsterminal bekommt Hilfsmethoden zum Einlesen, jede mit einem Parameter Scanner ein und einem Fragetext: `liesZahl` fragt so lange, bis eine ganze Zahl eingegeben ist; `liesUhrzeit` fragt so lange, bis `Format.minuten` die Eingabe annimmt; `liesRaum` fragt so lange, bis `campus.finde` einen Raum liefert. Lesen Sie jede Eingabe als ganze Zeile mit `nextLine()` und wandeln Sie sie selbst um. `Integer.parseInt` wirft bei vier eine `NumberFormatException`, und `NumberFormatException` ist eine Unterklasse von `IllegalArgumentException`.
- Gemeinsam: `main` legt einen Campus an mit LR-1 (Lernraum, 12 Plätze), LR-2 (Lernraum, 6), UX-LAB (Labor, 8), AUDIO-LAB (Labor, 4) und PR-1 (Raum, 30). Dann zeigt es das Menü 1 Räume 2 Buchen 3 Belegung 0 Ende, bis 0 gewählt wird. „Buchen“ fragt Raum, Gruppe, Personen, Tag, Beginn, Ende und „Einweisung vorhanden (j/n)“ ab und meldet, ob gebucht wurde. Wirft der Konstruktor von `Zeitfenster` oder `Buchung` eine `IllegalArgumentException`, zeigt das Terminal deren Meldung und kehrt ins Menü zurück.

Probieren Sie diese Eingaben: vier als Personenzahl, 10:00 als Beginn, XY als Kennung, eine leere Gruppe, ein Ende vor dem Beginn, 5 im Menü. Das Terminal meldet jede dieser Eingaben und fragt neu. Es stürzt bei keiner ab.

Drei Fehlerbilder, die in dieser Aufgabe häufig auftreten:

- `campus` als Attribut der Klasse statt als lokale Variable in `main` angelegt: `error: non-static variable campus cannot be referenced from a static context`. Geben Sie den `Campus` als Parameter an die Hilfsmethoden weiter.
- Personenzahl mit `nextInt()` gelesen und die Gruppe danach mit `nextLine()`: Nach der Eingabe 4, Enter, Team Blau erhält die Gruppe den leeren Text, und das Programm endet mit der folgenden Meldung (die Zeilennummer hängt von Ihrer Datei ab). Mit `nextInt()` und der Eingabe vier endet es mit `Exception in thread "main" java.util.InputMismatchException`.

```
Exception in thread "main" java.lang.IllegalArgumentException: Gruppe darf nicht leer sein
at freiraum.Buchung.<init>(Buchung.java:24)
```

- `campus.finde(kennung).belegungAusgeben()` ohne Prüfung auf null endet bei einer unbekannten Kennung mit einer `NullPointerException`, deren Meldung the return value of "`freiraum.Campus.finde(String)`" is null enthält.

Sechste Aufgabe (30 Minuten): Prüfen Sie, ob Labor hält, was Raum verspricht.

- Schreiben Sie über `Raum.buchen` einen Javadoc-Kommentar, der aufzählt, in welchen Fällen die Methode `false` liefert. Dieser Kommentar ist das Versprechen der Oberklasse an jeden Aufrufer.
- Schreiben Sie eine Klasse `Vertragsprobe` mit `main`. Sie legt mit `Raum r = new Labor("UX-LAB", 8);` ein Labor an, erzeugt eine Buchung ohne Einweisung für zwei Personen und eine Stunde und gibt `r.buchen(b)` aus.
- Beantworten Sie unter „Blatt 2 · Vertragsprobe“ in `NOTIZEN.md` drei Fragen in je einem Satz. Hält Labor das Versprechen aus dem Kommentar von `Raum.buchen`? An welcher Stelle Ihres Terminals würde ein Aufrufer, der nur Raum kennt, überrascht? Welche Alternative zur Unterklasse `Labor` sehen Sie? Nutzen Sie das Kriterium der Ersetzbarkeit aus T03B. Blatt 3 knüpft an diese drei Sätze an.

Siebte Aufgabe (30 Minuten): Ergänzen Sie `PruefungRaumtypen` um diese Fälle:

- Drei Buchungen hintereinander erhalten aufeinanderfolgende Nummern, und eine abgewiesene Buchung dazwischen verbraucht keine. Vergleichen Sie relativ zur ersten Nummer, etwa `b2.getNummer() == b1.getNummer() + 1`, denn der statische Zähler zählt über alle Buchungen eines Programmlaufs weiter.
- `Zeitfenster.aus("2", "10:00", "11:30")` ist equals zu `new Zeitfenster(2, 600, 690)`. `aus` mit "10:00" wirft eine `NumberFormatException`.
- Für Raum `r = new Labor("UX-LAB", 8)` liefert `toString()` den Text `Labor UX-LAB (8 Plätze)` und `maxDauerMinuten()` 240. Eine Buchung ohne Einweisung wird abgelehnt, mit Einweisung gelingt sie, 241 Minuten werden abgelehnt.
- In einem Lernraum gelingen 180 Minuten, 181 Minuten werden abgelehnt.
- `finde("XY")` liefert `null`, eine doppelte Kennung nimmt der Campus nicht auf.

Alle fünf Prüfprogramme enden mit 0 FEHLER. Mergen Sie nach `main`, setzen Sie den Tag mit `git tag abgabe-2`, pushen Sie ihn mit `git push origin abgabe-2` und packen Sie den Projektordner mit dem Ordner `.git` als ZIP.